

AT32 BLDC Hall 快速入门指南

前言

这篇应用笔记描述了如何快速使用电机开发板调试带霍尔传感器的 BLDC 电机运转的调适 SOP。雅特力有许多电机开发板，这篇文档以 AT-MOTOR-EVB 为例进行说明。

AT-MOTOR-EVB 支持型号列表：

支持型号	AT32F402/405 系列
	AT32F413 系列
	AT32F415 系列
	AT32F421 系列
	AT32F422/426 系列
	AT32F425 系列
	AT32L021 系列
	AT32F45x 系列
	AT32M412/M416 系列

目录

1	电机库算法概述.....	6
2	环境准备	7
2.1	硬件环境准备	7
2.2	软件环境准备	10
3	快速上手操作指南	14
3.1	修改电机控制模式及参数	15
3.2	建立连接	18
3.3	六步方波开环控制	18
3.4	霍尔相序自学习	20
3.5	确认电流取样点	21
3.6	参数自动鉴定	22
3.7	Q 轴电流调试	25
3.8	速度环控制	27
3.9	速度电压环调试	29
3.10	外部/软件命令控制	32
4	文档版本历史	34

表目录

表 1. 对应闪存存储空间 ROM 配置表.....	10
表 2. 宏定义设定(内部晶振).....	15
表 3. 宏定义设定(电机开发版号)	15
表 4. 宏定义设定(下臂 PWM 互补).....	15
表 5. 宏定义设定(下臂 MOS 反向输出)	15
表 6. 宏定义设定(反电势电压补偿).....	15
表 7. 宏定义设定(转子感测方式)	16
表 8. 宏定义设定(无电流环模式)	16
表 9. 宏定义设定(低速电压控制模式).....	16
表 10. 宏定义设定(电机线圈参数自动辨识).....	16
表 11. 宏定义设定(雅特力上位机).....	16
表 11. 宏定义设定(雅特力上位机).....	16
表 12. 电机参数宏定义	17
表 13. 文档版本历史	34

图目录

图 1. 电机开发板 AT-MOTOR-EVB	8
图 2. AT32F413RCT7 ROM 配置(Keil)	11
图 3. AT32F413RCT7 ROM 配置(AT32IDE).....	12
图 4. AT32F413RCT7 之 ROM 配置(IAR).....	13
图 5. 连接操作说明	18
图 6 控制模式选取开环控制	18
图 7.开环控制相关参数(BLDC)	19
图 8.开环控制示意图	19
图 9. 开环参数宏定义	19
图 10. 霍尔自学习.....	20
图 11. 霍尔自学习宏定义	21
图 12. 确认电流取样点信号测量位置(AT32-Motor-EVB2.0 为例)	21
图 13. 电流取样延迟时间测量波型	22
图 14. 电流取样点宏定义.....	22
图 15. 电机额定电流宏定义	23
图 16. 电机线圈参数自动辨识	23
图 17. 电机线圈参数宏定义	23
图 18. 电机额定电压	24
图 19. 电流 PI 控制参数自整定.....	24
图 20. 电流 PI 控制参数宏定义.....	24
图 21. 步阶电流示意图	25
图 22. 控制模式选取 IQ 电流环调试	25
图 23. PID 参数以及步阶电流参数	25
图 24. 调整通道监控参数(IQ 电流环调试)	26
图 25. 电流环调试波型	26
图 26. 电流 PI 控制参数宏定义.....	26
图 27. PID 参数以及加速度、减速度设置	27
图 28. 目标速度值设置	27
图 29. 调整信道监控参数(速度环调试).....	27
图 30. 速度环调试波型	28

图 31. 低速电压控制定义.....	29
图 32. 低速电压控制参数定义值.....	29
图 33. 速度电压 PID 参数设置.....	30
图 34. 低速的目标速度值设置	30
图 35. 调整通道监控参数(速度环调试).....	30
图 36. 速度电压环调试波型	31
图 37. 电位器位置图（外部电压控制来源）	32
图 38. 控制来源定义值(软件控制来源(上)/外部电压控制(下)).....	32
图 39. 速度控制模式(外部电压控制来源)	33
图 40 转矩控制模式(外部电压控制来源)	33
图 41 软件控制来源设置目标速度	33

1 电机库算法概述

这篇应用笔记使用电机库相关算法主要内容如下

目标电机：三相永磁同步电动机（直流无刷电动机）

控制模式：

- 120°方波控制

三相 PWM 调制模式：

- 120°导通 PWM 控制

相电流检测模式：

- 单电阻电流检测和重构方式

转子位置检测模式：

- 霍尔效应位置传感器

有传感器 120°方波控制模式：

- 120°方波电压控制
- 转矩控制 (120°方波电流控制)
- 转速控制
- 正逆转控制

自动调试功能：

- 霍尔状态自学习(霍尔传感器专用)
- 电机线圈参数自动辨识
- 电流 PI 控制参数自整定

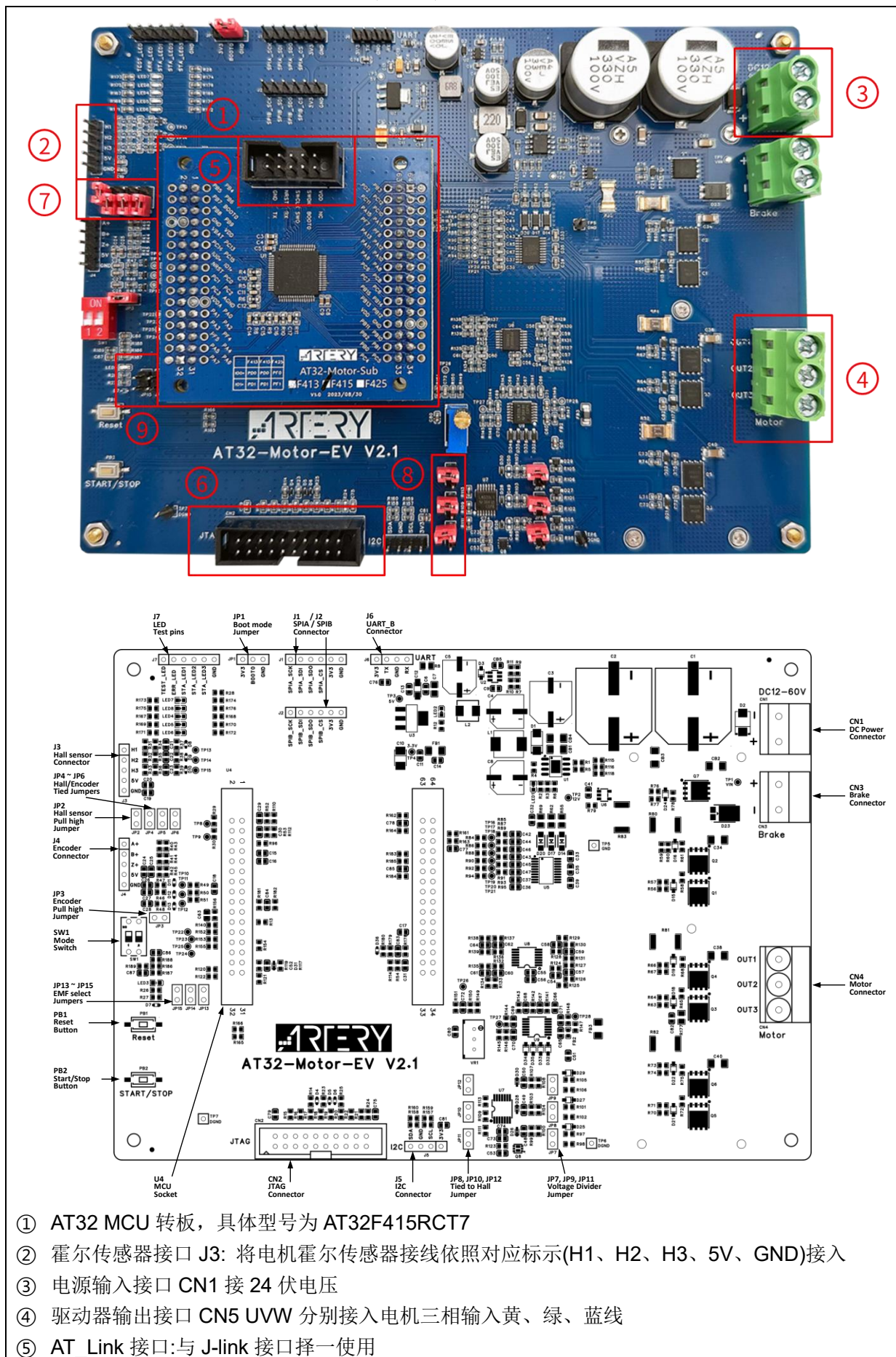
2 环境准备

2.1 件环境准备

需要准备硬件项目主要包括 PMSM(BLDC)电机、AT-Link 或第三方调试下载器以及一块电机开发控制板 AT-MOTOR-EVB，相关硬件配置可参考 UM0011 低压电机控制开发板使用手册。

- PMSM(BLDC)电机
- 调试下载器
- 电机控制开发板: AT-MOTOR-EVB (含 AT32F415RCT7 转板)

图 1. 电机开发板 AT-MOTOR-EVB



- ⑥ J-link 接口:与 AT_Link 接口择一使用
- ⑦ 跳线设定: JP2 CLOSED、JP2、JP4、JP5、JP6 OPEN
- ⑧ 跳线设定: JP8、JP10、JP12 OPEN
- ⑨ 跳线设定: JP13、JP14、JP15 OPEN

2.2 软件环境准备

- 1) 开启 bldc_1shunt_hall_sensor 范例工程
- 2) 电机应用 PC 软件 ArteryMotorMonitor.exe（本软件不需安装，只需直接运行可执行程序）。
- 3) Keil 的配置需根据各个 AT32 MCU 的闪存存储大小修改 Options 中的 Read/Only Memory Areas，详细参照表 1，例：AT32F413RCT7 的闪存存储大小为 256 K 字节，则其 IROM1 的起始位置为 0x8000000，大小为 0x3F800，其 IROM2 的起始位置为 0x803F800，大小为 0x800，AT32F413RCT7 的修改范例如图 2 所示；AT32IDE 的配置需根据各个 AT32 MCU 的闪存存储大小修改 Id 文件如图 3 所示；IAR Systems 的配置需根据各个 AT32 MCU 的闪存存储大小修改 icf 文件如图 4 所示。

表 1. 对应闪存存储空间 ROM 配置表

Flash size	4032K	1024K	512K	448K	256K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x3EF000	0xFF800	0x7F800	0x6F000	0x3F800
IROM2(address)	0x83EF000	0x80FF800	0x807F800	0x0806F000	0x803F800
IROM2(size)	0x1000	0x800	0x800	0x1000	0x800

Flash size	128K	64K	32K	16K
IROM1(address)	0x8000000	0x8000000	0x8000000	0x8000000
IROM1(size)	0x1FC00	0x0FC00	0x07C00	0x03C00
IROM2(address)	0x801FC00	0x800FC00	0x8007C00	0x8003C00
IROM2(size)	0x400	0x400	0x400	0x400

注[1]:使用 keil v5.33 版本，因 AT32 BSP 源码不支持 V6.15 编译器，请使用 keil complier version 5 版本进行编译。

图 2. AT32F413RCT7 ROM 配置(Keil)

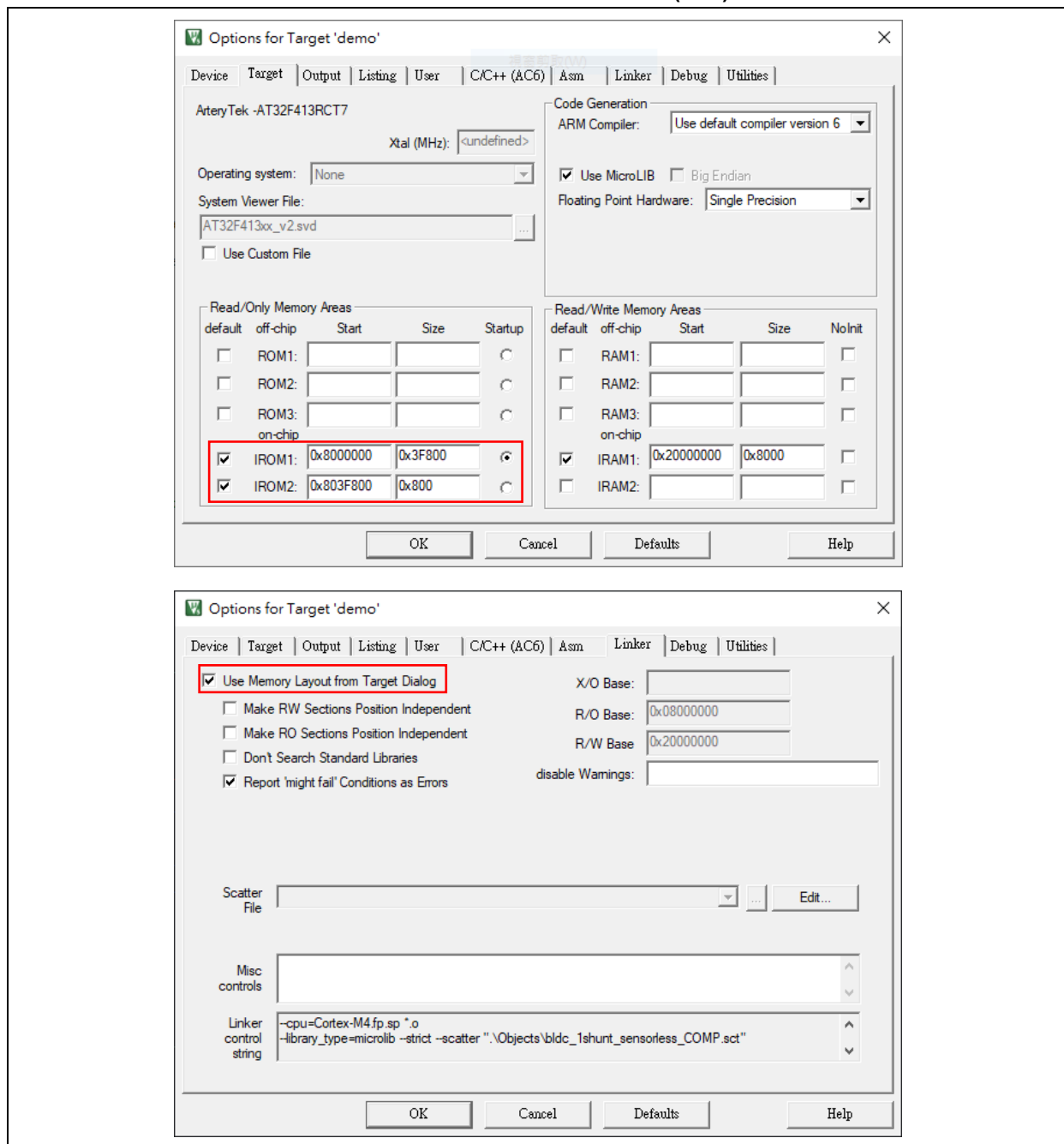


图 3. AT32F413RCT7 ROM 配置(AT32IDE)

```
AT32F413xC_FLASH.ld X
25 _estack = 0x20008000; /* end of RAM */
26
27 /* Generate a link error if heap and stack don't fit into RAM */
28 _Min_Heap_Size = 0x200; /* required amount of heap */
29 _Min_Stack_Size = 0x400; /* required amount of stack */
30
31 /* Specify the memory areas */
32 MEMORY
33 {
34     FLASH (rx)      : ORIGIN = 0x08000000, LENGTH = 0x3F800
35     MC_DATA (r)      : ORIGIN = 0x0803F800, LENGTH = 0x800
36     RAM (xrw)        : ORIGIN = 0x20000000, LENGTH = 32K
37 }
38
39 /* Define output sections */
40 SECTIONS
41 {
42     /* The startup code goes first into FLASH */
43     .isr_vector :
44     {
45         . = ALIGN(4);
46         KEEP(*(.isr_vector)) /* Startup code */
47         . = ALIGN(4);
48     } >FLASH
49
50     .mc_data :
51     {
52         . = ALIGN(4);
53         . = ORIGIN(MC_DATA);
54         _MC_VectStoreAddr1 = .;
55         *(.MC_VectStoreAddr1);
56         . = ALIGN(4);
57         . = ORIGIN(MC_DATA) + 800;
58         _MC_VectStoreAddr2 = .;
59         *(.MC_VectStoreAddr2);
60         . = ALIGN(4);
61     } >MC_DATA
62
63     /* The program code and other data goes into FLASH */
64     .text :
65     {
66         . = ALIGN(4);
67         *(.text)           /* .text sections (code) */
68         *(.text*)          /* .text* sections (code) */
69         *(.glue_7)          /* glue arm to thumb code */
70         *(.glue_7t)         /* glue thumb to arm code */
71         *(.eh_frame)
72     }
```

图 4. AT32F413RCT7 之 ROM 配置(IAR)

```

1  /**IcfEditorFile="$TOOLKIT_DIR$\config\ide\IcfEditor\cortex_v1_0.xml" */
2  /*-Editor annotation file-*/
3  /*-Specials-*/
4  define symbol __ICFEDIT_intvec_start__ = 0x08000000;
5  /*-Memory Regions-*/
6  define symbol __ICFEDIT_region_ROM_start__ = 0x08000000;
7  define symbol __ICFEDIT_region_ROM_end__ = 0x0803F800 - 1;
8  define symbol __ICFEDIT_region_ROM1_start__ = 0x0803F800;
9  define symbol __ICFEDIT_region_ROM1_end__ = (__ICFEDIT_region_ROM1_start__ + 800 - 1);
10 define symbol __ICFEDIT_region_ROM2_start__ = (__ICFEDIT_region_ROM1_end__ + 1);
11 define symbol __ICFEDIT_region_ROM2_end__ = 0x0803F800 + 0x800 - 1;
12 define symbol __ICFEDIT_region_RAM_start__ = 0x20000000;
13 define symbol __ICFEDIT_region_RAM_end__ = 0x20007FFF;
14 /*-Sizes-*/
15 define symbol __ICFEDIT_size_cstack__ = 0x400;
16 define symbol __ICFEDIT_size_heap__ = 0x200;
17 /*** End of ICF editor section. ***ICF*** */
18
19 define memory mem with size = 4G;
20 define region ROM_region = mem:[from __ICFEDIT_region_ROM_start__ to __ICFEDIT_region_ROM_end__];
21 define region ROM1_region = mem:[from __ICFEDIT_region_ROM1_start__ to __ICFEDIT_region_ROM1_end__];
22 define region ROM2_region = mem:[from __ICFEDIT_region_ROM2_start__ to __ICFEDIT_region_ROM2_end__];
23 define region RAM_region = mem:[from __ICFEDIT_region_RAM_start__ to __ICFEDIT_region_RAM_end__];
24
25
26 define block CSTACK with alignment = 8, size = __ICFEDIT_size_cstack__ { };
27 define block HEAP with alignment = 8, size = __ICFEDIT_size_heap__ { };
28
29 initialize by copy { readwrite };
30 do not initialize { section .noinit };
31
32 place at address mem: __ICFEDIT_intvec_start__ { readonly section .intvec };
33
34
35 place in ROM_region { readonly };
36 place in ROM1_region { readonly section .MC_VectStoreAddr1 };
37 place in ROM2_region { readonly section .MC_VectStoreAddr2 };
38 place in RAM_region { readwrite,
39 block CSTACK, block HEAP };

```

3 快速上手操作指南

BLDC 六步方波霍尔传感器控制调试流程如下：

步骤 1. 根据电机参数以及应用修改电机控制模式及参数(参考章节 3.1)

步骤 2. 与 UI 电机控制软件建立连接(参考章节 3.2)

步骤 3. 使用开环控制确认电机是否可正常运转(参考章节 3.3)

步骤 4. 霍尔相序自学习(参考章节 3.4)

步骤 5. 电流环控制参数调整

(如不需要电流环，在步骤 1.修改电机控制模式下开启宏定义 WITHOUT_CURRENT_CTRL，并略过此步骤 5 及步骤 6，直接跳至步骤 7 调整速度电压环)

STEP 1 - 用示波器确认电流取样点(参考章节 3.5)

STEP 2 - 电机线圈参数自动鉴定(参考章节 3.6)

STEP 3 - 电流 PI 控制参数自整定(参考章节 3.6)

STEP 4 - IQ tune 微调电流环 Kp Ki 参数(参考章节 3.7)

步骤 6. 速度控制参数调整(参考章节 3.8)

步骤 7. 速度电压控制参数调整(参考章节 3.9)

步骤 8. 预设为通过 UI 软件给定控制命令，如需修改为外部电位器控制，可调整控制命令来源(参考章节 3.10)

3.1 修改电机控制模式及参数

- 在电机库文档中的 `motor_control_drive_param.h` 内提供用户自行输入电机控制模式、电机参数、控制板参数、控制器参数。
- 模式宏定义基于用户的硬件与电机特性，可定义适当的模式，以下说明宏定义设定步骤：

- 根据用户需要选择是否使用内部晶振

表 2. 宏定义设定(内部晶振)

宏定义名称	描述
INTERNAL_CLOCK_SOURCE	使用内部晶振(默认为不开启)

- 如使用的电机开发板(如 AT32-Motor-EV)有多个版本，则需根据用户电机开发版选择使用的版本(择一开启)

表 3. 宏定义设定(电机开发版号)

宏定义名称	描述
AT_MOTOR_EVB_V1	使用电机开发板 AT-MOTOR-EVB V1.x (include <code>mc_hwio_v1.h</code> 文档)
AT_MOTOR_EVB_V2	使用电机开发板 AT-MOTOR-EVB V2.0 (include <code>mc_hwio_v1.h</code> 文档)

- 根据用户需要的控制方法选择是否开启下臂 MOS 管开关与上臂 PWM 互补,无传感器时建议开启互补模式

表 4. 宏定义设定(下臂 PWM 互补)

宏定义名称	描述
COMPLEMENT	开启下臂 MOS 管开关与上臂 PWM 互补(默认为开启, 若注解掉则此功能关闭)

- 根据用户硬件电路上使用的 Gate driver 型号是否有下臂反向输出功能决定是否开启此功能, 以 AT32-Motor-EV 上使用的 U3315 为例有反向, 因此预设为开启; AT32M412-LV-Motor-EV 使用的 FD6288Q 为例无反向, 因此预设为关闭

表 5. 宏定义设定(下臂 MOS 反向输出)

宏定义名称	描述
GATE_DRIVER_LOW_SIDE_INVERT	开启下臂 MOS 反向输出(AT32-Motor-EV 默认为开启, AT32M412-LV-Motor-EV 默认为关闭)

- 根据用户电机特性决定是否开启反电势电压补偿功能

表 6. 宏定义设定(反电势电压补偿)

宏定义名称	描述
EMF_COMPENSATE	反电势电压补偿(默认为不开启, 若定义则此功能开启)

- 本专案为方波霍尔传感器范例工程, 预设为霍尔传感器, 用户无需修改, 但因代码判断需求需保留此选项

表 7. 宏定义设定(转子感测方式)

宏定义名称	描述
HALL_SENSORS	霍尔传感器
SENSORLESS	无传感器控制的模式(通用)

7. 根据用户是否需要电流环控制选择是否开启

表 8. 宏定义设定(无电流环模式)

宏定义名称	描述
WITHOUT_CURRENT_CTRL	纯电压控制,无电流环(默认为不开启, 若定义则此功能开启)

8. 根据用户是否需要低速电压控制选择是否开启

表 9. 宏定义设定(低速电压控制模式)

宏定义名称	描述
LOW_SPEED_VOLT_CTRL	低速电压控制模式(默认为开启, 若注释掉则此功能不开启)

9. 电机线圈参数自动辨识功能, 打开后可自动检测电机线圈 RS、LS 以及计算电流环 PI 控制参数(详细说明见章节 3.6)

表 10. 宏定义设定(电机线圈参数自动辨识)

宏定义名称	描述
MOTOR_PARAM_IDENTIFY	电机线圈参数自动辨识(默认为开启, 若注解掉则此功能关闭)

10. 根据用户是否需要使用雅特力上位机选择是否开启

表 11. 宏定义设定(雅特力上位机)

宏定义名称	描述
USE_MOTOR_MONITOR	使用雅特力上位机(默认为关闭, 若注释掉则此功能开启)

11. 根据用户需要选择是否开启电流滤波功能

表 12. 宏定义设定(电流滤波功能)

宏定义名称	描述
CURRENT_LP_FILTER	开启电流滤波功能(默认为开启, 若定义则此功能开启)

- 相关电机参数的宏定义如表 13 所示, 如极对数、编码器参数、霍尔传感器参数等。在此仅列出此工程范例相关参数。

表 13. 电机参数宏定义

宏定义名称	描述
RS_LL	电机线间电阻值(unit: Ω)
LS_LL	电机线间电感值(unit: H)
POLE_PAIRS	极对数
KE	电机 KE 值
NOMINAL_CURRENT	电机额定电流 (unit: ampere)
HALL_LEARN_DIR	HALL 自学习后的电机转向
HALL_LEARN_0_STATE	HALL 自学习后的第一个状态(BLDC: 输出 V 相上臂 PWM 与 W 相下臂 ON)
HALL_LEARN_1_STATE	HALL 自学习后的第二个状态(BLDC: 输出 V 相上臂 PWM 与 U 相下臂 ON)
HALL_LEARN_2_STATE	HALL 自学习后的第三个状态(BLDC: 输出 W 相上臂 PWM 与 U 相下臂 ON)
HALL_LEARN_3_STATE	HALL 自学习后的第四个状态(BLDC: 输出 W 相上臂 PWM 与 V 相下臂 ON)
HALL_LEARN_4_STATE	HALL 自学习后的第五个状态(BLDC: 输出 U 相上臂 PWM 与 V 相下臂 ON)
HALL_LEARN_5_STATE	HALL 自学习后的第六个状态(BLDC: 输出 U 相上臂 PWM 与 W 相下臂 ON)

3.2 建立连接

在确保完成硬件准备和软件环境准备后，我们可以进行以下操作建立 UI 与控制版的连接。详细电机 UI 使用说明请参照 AN0063 文档。

STEP-1

正确将电机、AT_Link/Jlink、开发板电源接到电机开发板上，将 USART 接口同 USB 线接入 PC。

STEP-2

使用 MDK 编译 demo 工程代码，使用 Jlink 或 AT_Link 下载到开发板的芯片中。

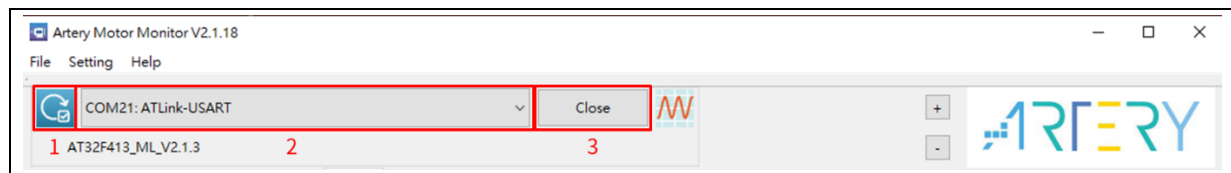
STEP-3

执行程序 ArteryMotorMonitor_V2.1.x.exe(V2.1.x 为软件版本号)，在 File -> Open Porject 选项中选择 ArteryMotorMonitor_V2.1.x.atmcx->开启。

STEP-4

点选 Serial Port 更新图标(1.)并选取对应的串口号(2.)，点击 Open(3.)即可开启串口实时通信,操作步骤如图 5。

图 5. 连接操作说明

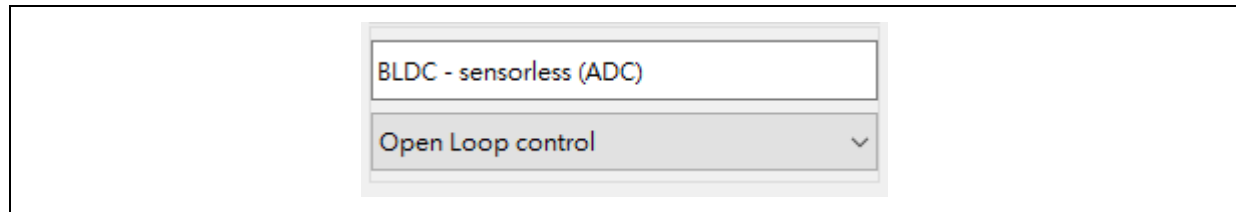


3.3 六步方波开环控制

此模式使用开环电压控制模式，不需位置传感器即可转动电机，用来确认电机是否可正常运转与运转方向是否正确或用来调整 BLDC 无传感器开环启动参数。开环电压与速度递增量的调试方式可视电机运转速度与电机相电流大小调整。

STEP-1. 将控制模式下拉菜单选为 Open Loop control

图 6 控制模式选取开环控制

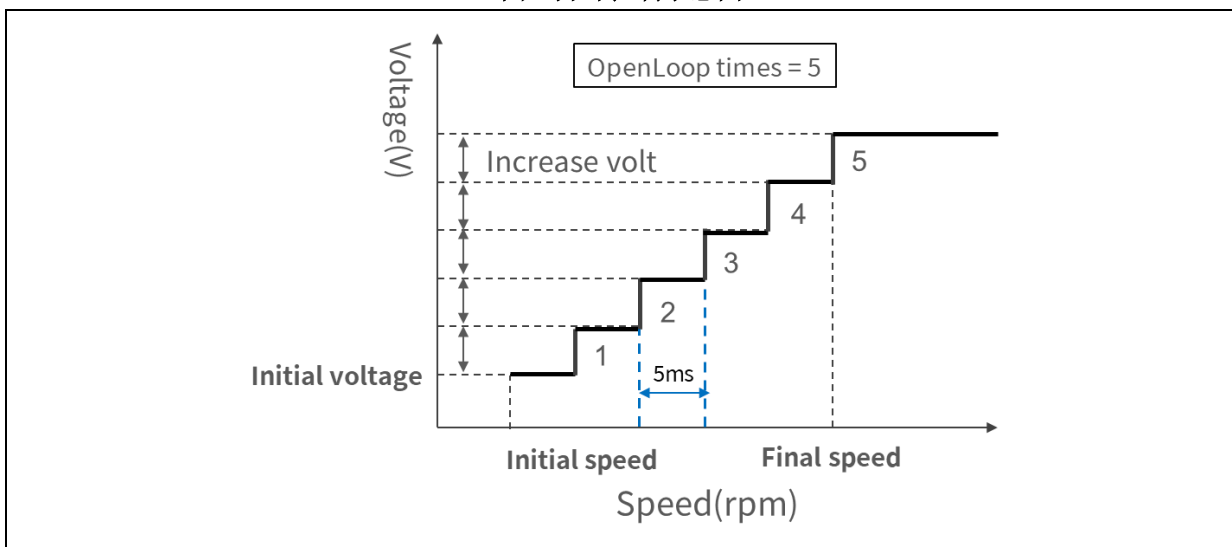


STEP-2. 切换到 Tuning Parameters 页面(如图 7)，调整初始电压(OpenLoop initial voltage)、初始速度(OpenLoop initial speed)、最终速度(OpenLoop final speed)以及递增次数(OpenLoop times)、每次递增电压(OpenLoop increase volt)，开环控制的示意图如图 8。

图 7.开环控制相关参数(BLDC)

Basic Tuning Parameters		
Open Loop Control		
OpenLoop initial voltage	0.75	V
OpenLoop initial speed	100	rpm
OpenLoop increase volt	0.004	V
OpenLoop final speed	350	rpm
OpenLoop times	90	

图 8.开环控制示意图



调试技巧:

1. 刚开始可以先将 Openloop increase volt, Openloop times 调整为 0, Openloop final speed 与 Openloop initial speed 调整成一样的数值, 先找到适当的起始电压及速度
2. 找到适当起始电压后, 进一步设定最终速度, 并调整 increase volt 以及 Openloop times, 若觉得运转无力则调大 increase volt 增加每次递增电压或 Openloop times 增加递增次数(一般来说建议递增次数可以大于 50 次)

STEP-3. 按下启动电机(Start Motor)按钮, 观察电流大小以及电机运转情况, 直到电机正常运转。(开环电压避免过大以免造成电机过热损毁)

STEP-4. 将调试后参数填入 motor_control_drive_param.h 文件宏定义, 如图 9 所示, 并重新编译烧录代码。

图 9. 开环参数宏定义

```

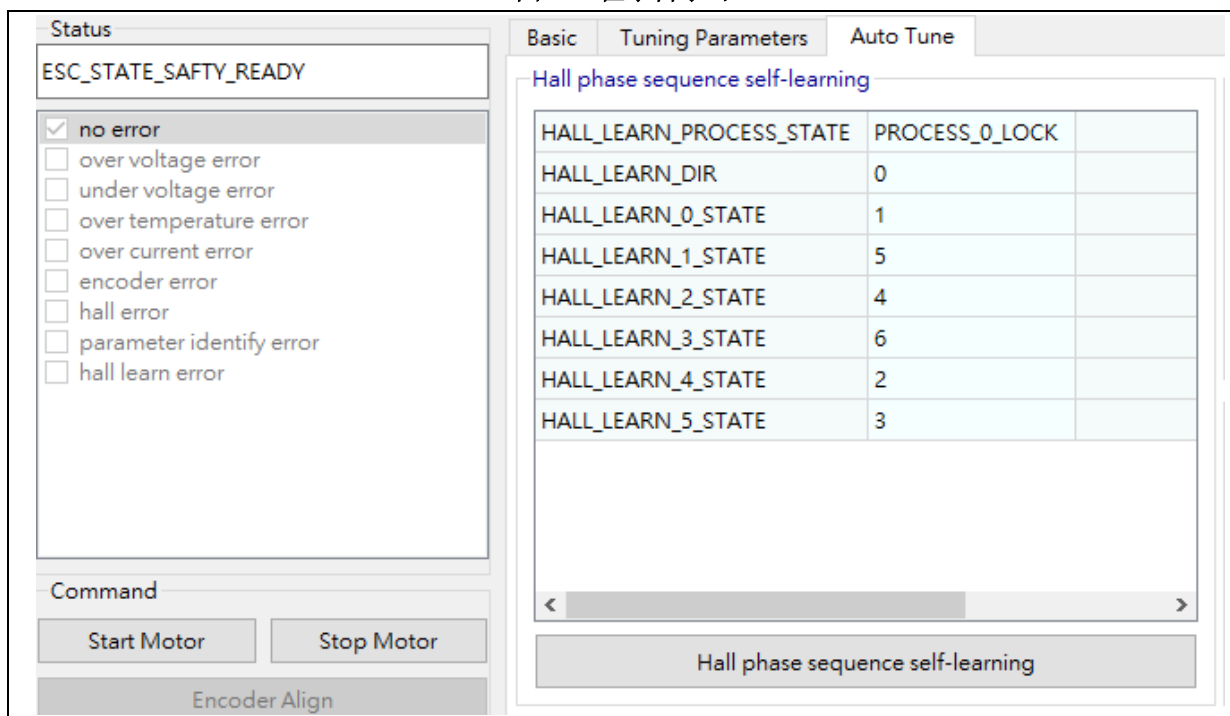
motor_control_drive_param.h
225  /* open loop control */
226  #define OLC_INIT_SPD           (100)      /*!< rpm */
227  #define OLC_FINAL_SPD         (350)      /*!< rpm */
228  #define OLC_TIMES              (90)       /*!< Accumula
229  #define OLC_INIT_VOLT          (0.75)     /*!< V */
230  #define OLC_VOLT_INC           (0.004)
    
```

3.4 霍尔相序自学习

STEP-1. 连接 UI 软件。

STEP-2. 进入调试模式，切换到 Auto Tune 页面，在状态机的状态为 ESC_STATE_SAFETY_READY 时按下 Hall phase sequence self-learning 按钮即执行霍尔相序自学习，如图 10 所示。

图 10. 霍尔自学习



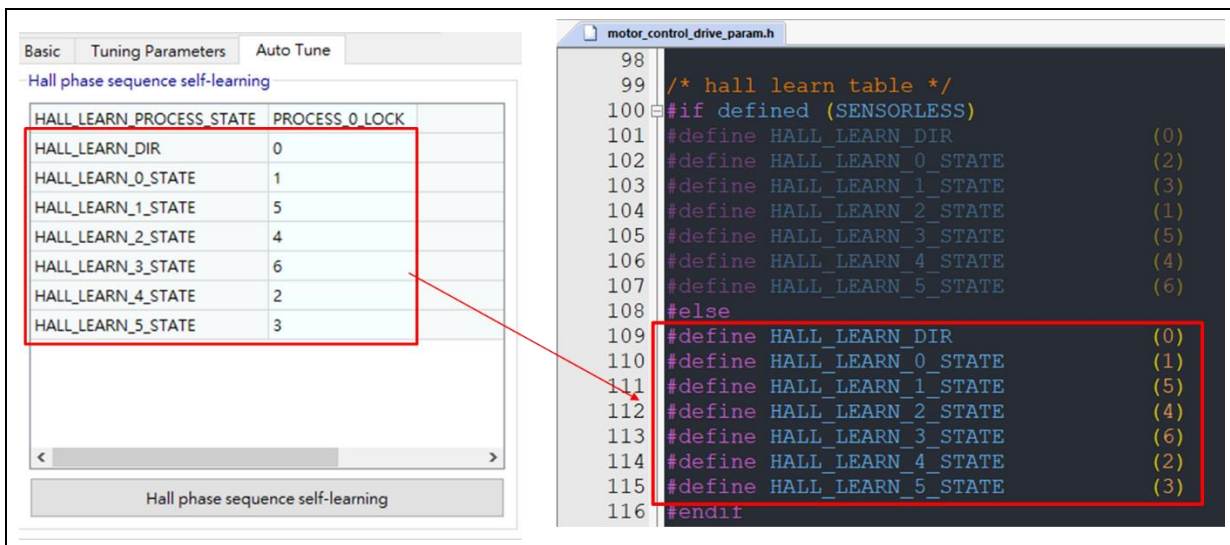
STEP-3. 电机开环运转，此时若电机转向相反，待电机停下时，再重新按下 Hall phase sequence self-learning 按钮即可将转向颠倒，等待电机直到停止运转则为学习完成。

注:若电机运转不顺或无法运转，可以回到开环控制调整开环电压以及开环运转速度宏定义并重新编译烧录，再重新执行 STEP-2。

STEP-4. 观察 HALL_LEARN_PROCESS_STATE 状态是否为 PROCESS_4_FINISH

STEP-5. 填入 HALL_LEARN_DIR 与 HALL_LEARN_0_STATE~HALL_LEARN_5_STATE 数值至“hall learn table”宏定义(motor_control_drive_param.h) (图 11)并重新编译烧录代码。

图 11. 霍尔自学习宏定义



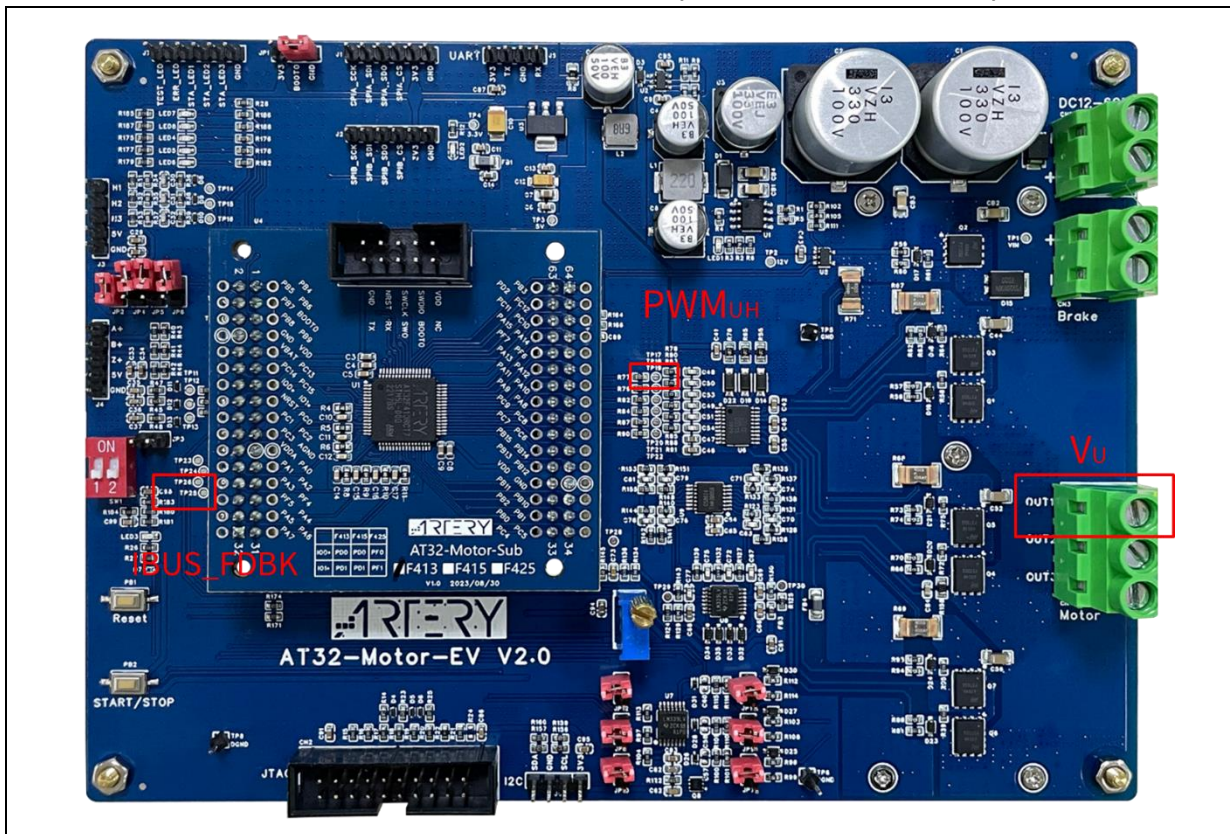
3.5 确认电流取样点

为了确认电流取样点，需要准备一台至少有 3 通道的示波器，测量的信号如下(图 12):

示波器通道	测量位置	AT32-Motor-EVB2.0 为例
CH1	U 相输出电压	CN4 OUT1
CH2	电流检测信号 *注[3]	IBUS_FDBK(TP26)
CH3	PWM UH 输出信号(MCU 端)	TIM1_CH1(TP17)

*注[3].因需检测正负电流，因此电流 0A 时电流检测信号有一个直流偏置，以 AT32-Motor-EVB2.0 为例为 0.833V，信号测量时 CH2 需要设置消零(-0.83V)

图 12. 确认电流取样点信号测量位置(AT32-Motor-EVB2.0 为例)



STEP-1. 使用六步方波开环控制使电机运转(尽量运转在低速)

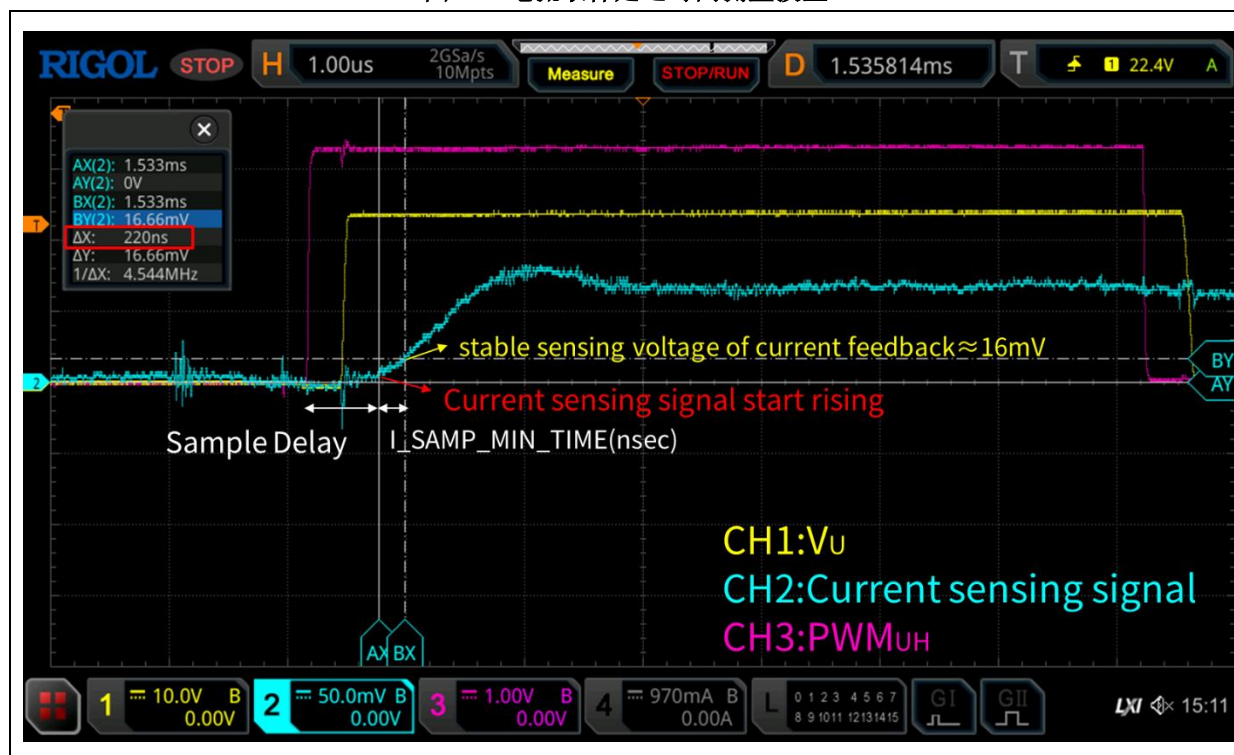
STEP-2. 以 PWM_UH 上升沿当作触发点, 测量电流取样延迟时间, 如图 12

Sample Delay = PWM UH 上升到电流检测信号开始上升的时间差(unit:usec)

$I_SAMP_DELAY(nsec) = Sample\ Delay(nsec) + Dead\ time(nsec)$

STEP-3. 将 Cursor 移至稳定的 ADC 取样电压(约 16mV), 测量 CH2 电流检测信号从 0mV 到 16mV 所需时间即为 I_SAMP_MIN_TIME 时间, 以图 13 为例为 220 nsec

图 13. 电流取样延迟时间测量波形



STEP-4. 将 STEP-3.测量得到的延迟时间填入 motor_control_drive_param.h 文件宏定义 I_SAMPLE_MIN_TIME, 如图 14 所示, 并重新编译烧录代码。

图 14. 电流取样点宏定义

```
motor_control_drive_param.h
196 /* I-SAMPLE PARAMETER */
197 #define I_SAMP_MIN_TIME          (220)          /*!< nsec */
198 #define I_SAMP_DELAY              (500)          /*!< nsec */
```

3.6 参数自动鉴定

参数自动鉴定分为两部分, 分别为电机线圈参数自动辨识、电流 PI 控制参数自整定。

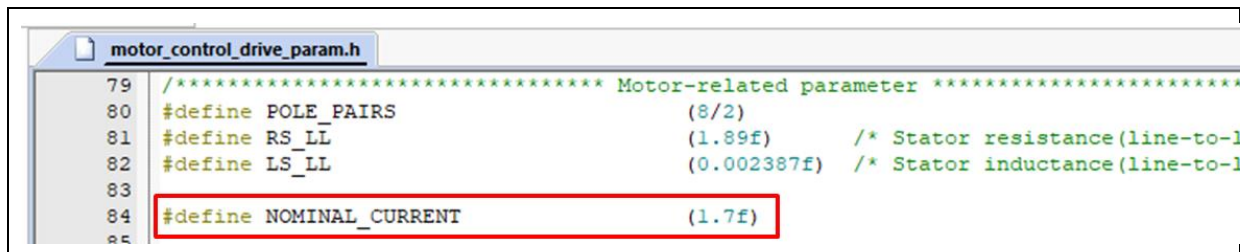
若已有电机 RS_LL 以及 LS_LL 参数则不需进行电机线圈参数自动辨识, 可以直接修改 motor_control_drive_param.h 文件内的宏定义 RS_LL 以及 LS_LL。

电机线圈参数自动辨识:

STEP-1. 在 motor_control_drive_param.h 文件内, 填入电机额定电流(NOMINAL_CURRENT)宏定

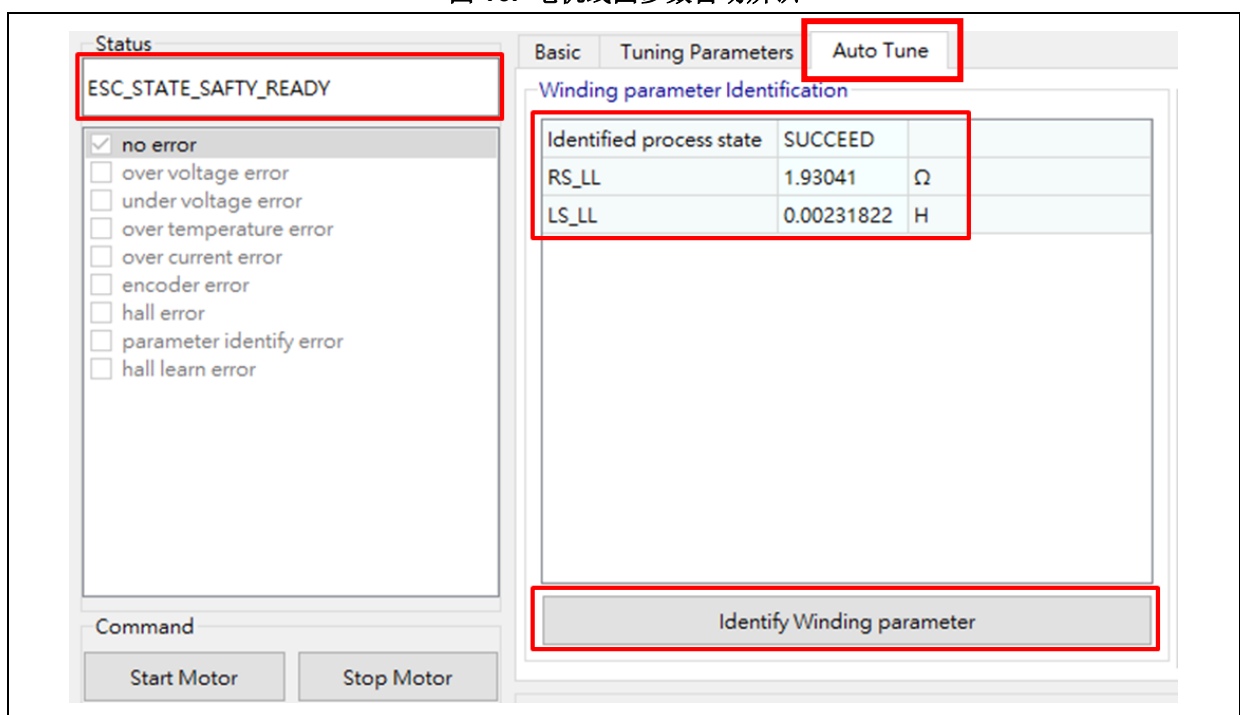
义并重新编译烧录代码，如图 15 所示。

图 15. 电机额定电流宏定义



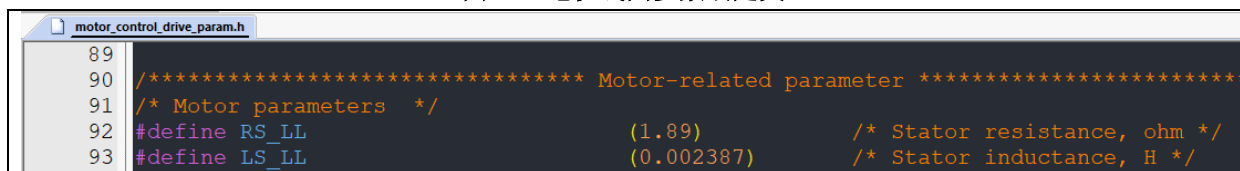
STEP-2. 使用 UI 软件，切换到 Auto Tune 页面，在状态机的状态为 safty ready 时按下 Identify Winding parameter 按钮即执行电机线圈参数自动辨识，如图 16 所示。

图 16. 电机线圈参数自动辨识



STEP-3. Identified process state 回报是否成功(SUCCEED)并获得 RS_LL 以及 LS_LL 参数并将数值填入 motor_control_drive_param.h 文件宏定义 RS_LL 以及 LS_LL 参数，如图 17 所示。

图 17. 电机线圈参数宏定义



电流 PI 控制参数自整定：

STEP-1. 在 motor_control_drive_param.h 文件内，填入输入电源 DC 电压(VDC_RATED)宏定义并重新编译烧录代码，如图 18 所示。

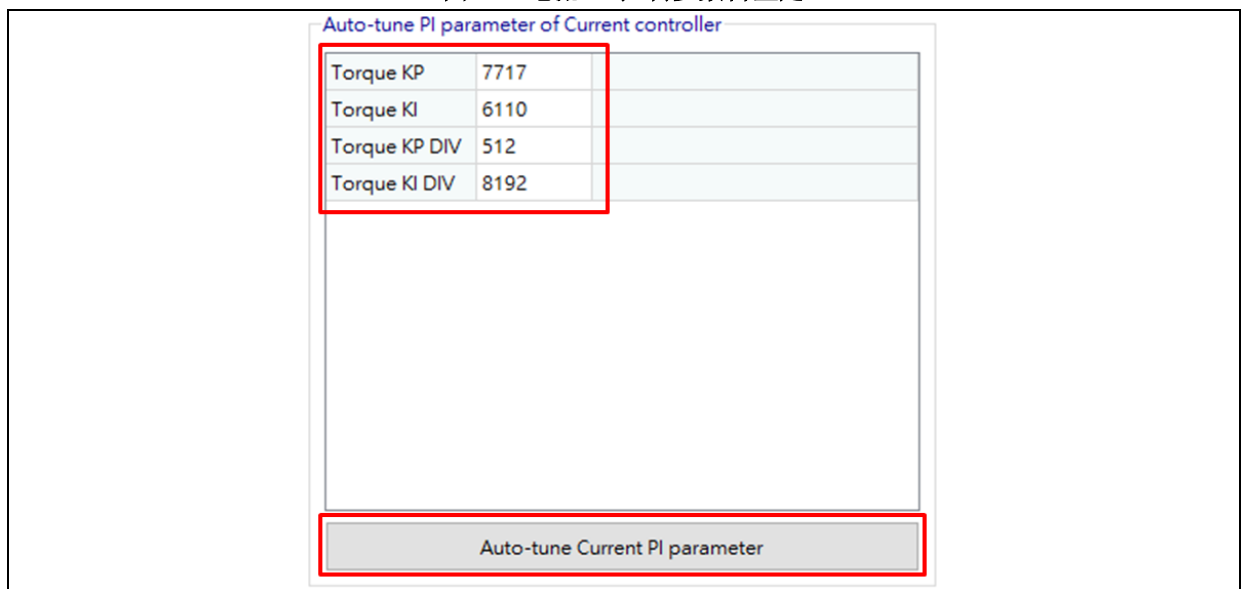
图 18. 电机额定电压

```

motor_control_drive_param.h
100
101 /***** Drive-related parameter *****/
102 /* basic */
103 #define VDC_RATED (24.0f)
104 #define V_SENSE_GAIN (10/(3.9f+180+10)) // 0.05157
105 #define ADC_REFERENCE_VOLT (3.3f)
106 #define ADC_DIGITAL_SCALE_12BITS (4095)
    
```

STEP-2.切换到 Auto Tune 页面，在状态机的状态为 safty ready 时按下 UI 软件的 Auto-tune Current PI parameter 按钮即执行电流 PI 控制参数自整定，完成后会自动更新 Q 轴电流的 PI 控制参数，如图 19 所示。

图 19. 电流 PI 控制参数自整定



STEP-3.将图 19 显示的参数填回宏定义(motor_control_drive_param.h) 并重新编译烧录代码，如图 20。

图 20. 电流 PI 控制参数宏定义

```

motor_control_drive_param.h
242 /* pi parameter */
243 #define PID_IS_KP_DEFAULT 2000
244 #define PID_IS_KI_DEFAULT 200
245 #define PID_IS_KP_DIV 256
246 #define PID_IS_KP_DIV_LOG LOG2(PID_IS_KP_DIV)
247 #define PID_IS_KI_DIV 1024
248 #define PID_IS_KI_DIV_LOG LOG2(PID_IS_KI_DIV)
    
```

旧版电机库(MC_Library_Project_V2.1.1 以前)的换算公式如下:

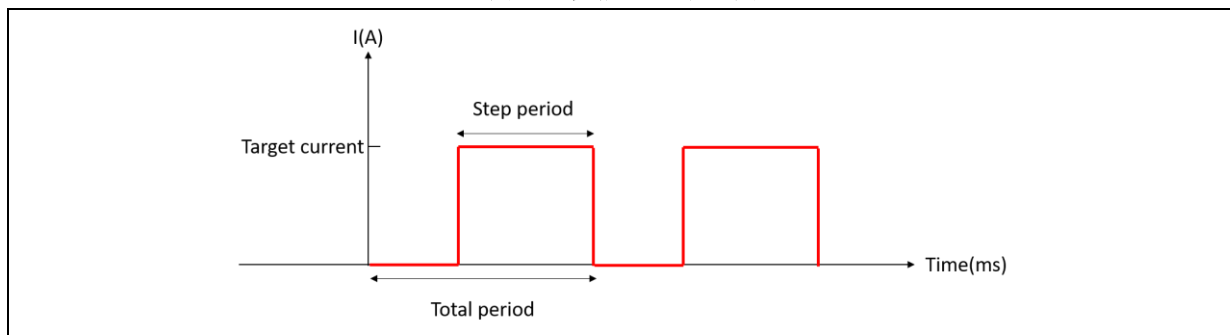
$$PID_IS_KP_DIV_LOG = \log_2 \text{Torque KP DIV}$$

$$PID_IS_KI_DIV_LOG = \log_2 \text{Torque KI DIV}$$

3.7 Q 轴电流调试

此模式下会产生一个步阶的电流，如图 21，可以调整步阶电流的相关参数，产生步阶电流的目的是为了查看调整 Q 轴电流的 PID 电流环参数后的电流响应。

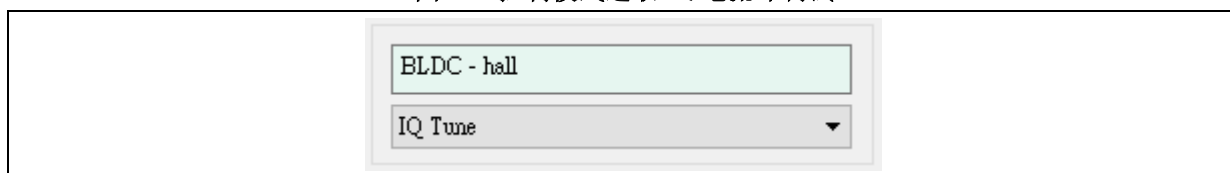
图 21. 步阶电流示意图



详细操作步骤如下：

STEP-1. 将控制模式下拉菜单选为 IQ tune

图 22. 控制模式选取 IQ 电流环调试



STEP-2. 切换到 Tuning Parameters 页面，设置 PID 参数以及步阶电流参数，初次调整可以使用电流 PI 控制参数自整定后的参数，如电流响应不如预期在进行调整，如图 23

图 23. PID 参数以及步阶电流参数

Basic

Tuning Parameters

Unit step config

Current Tune target current	0.999	A
Current Tune total period	100	ms
Current Tune step period	2	ms

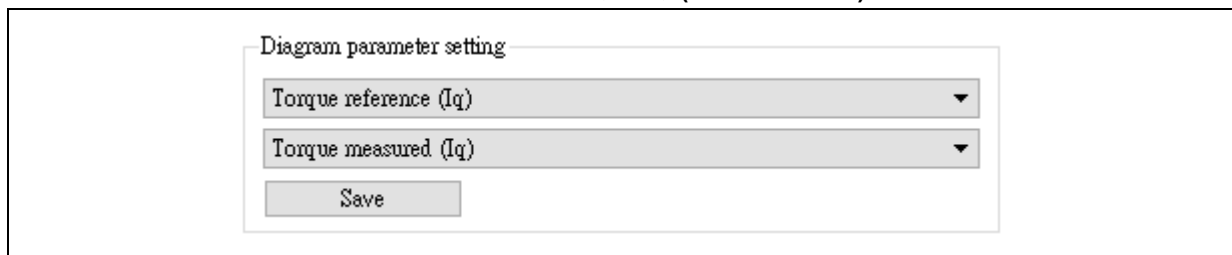
IQ Current control

Torque KP	25000	
Torque KI	3000	
Torque KP DIV	2048	
Torque KI DIV	4096	

STEP-3. 按下启动电机(Start Motor)按钮

STEP-4. 调整绘图区的参数监控为 Torque reference(Iq)以及 Torque measured(Iq)并按下 Save 键

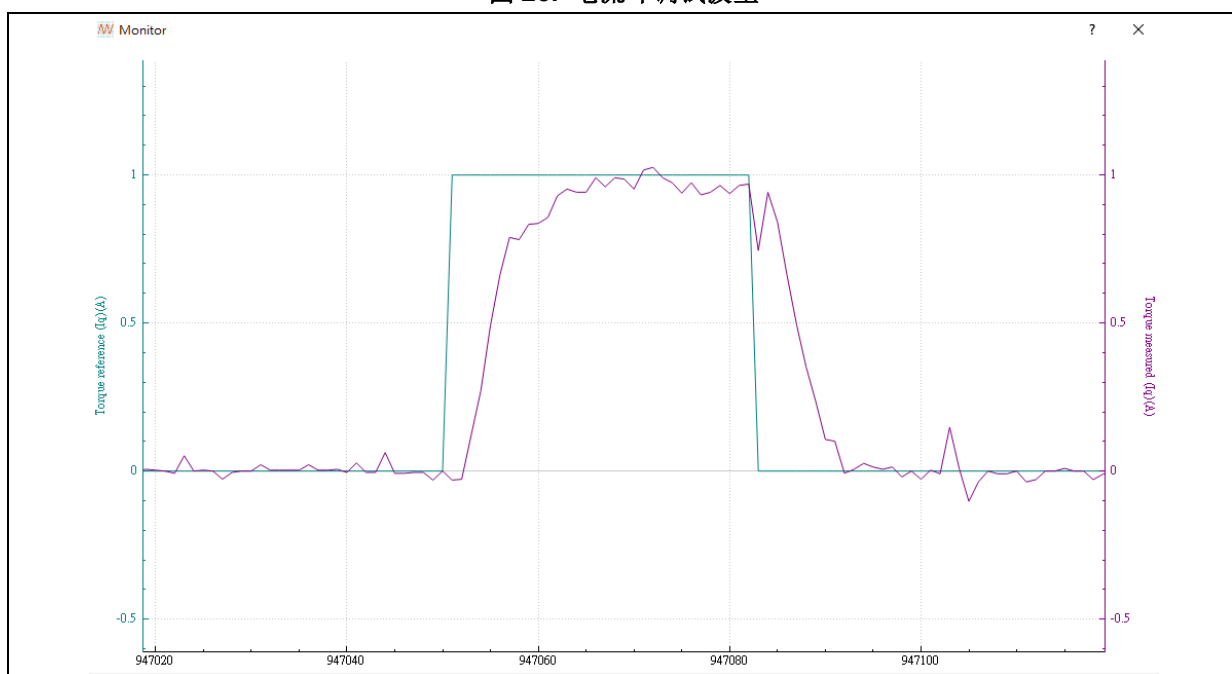
图 24. 调整通道监控参数(IQ 电流环调试)



STEP-5. 点选绘图组  即可呼叫出波形窗口，波形制图控制按钮详细说明可参考 AN0063 AT32 Motor Monitor Application Note

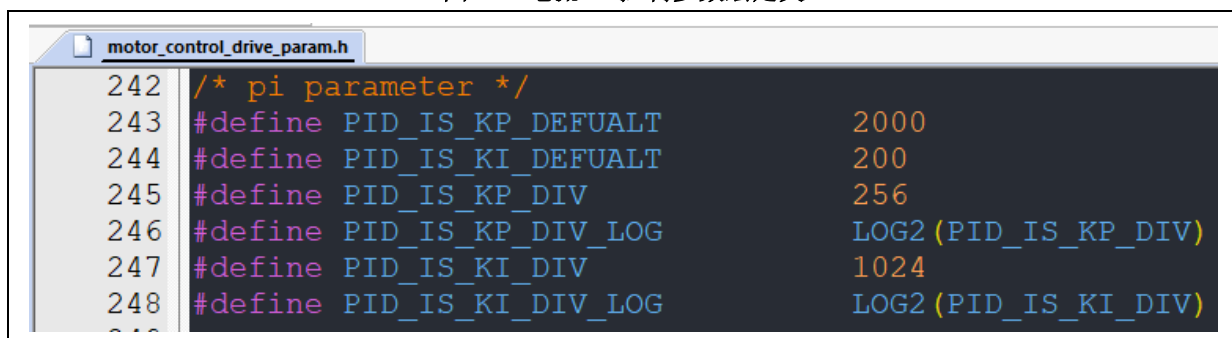
STEP-6. 查看电流响应是否如预期，如图 25，若不如预期则按下停止电机，并重复 STEP-2~STEP-6

图 25. 电流环调试波形



STEP-7. 微调完成后将参数填回宏定义(motor_control_drive_param.h) 并重新编译烧录代码，如图 26。

图 26. 电流 PI 控制参数宏定义



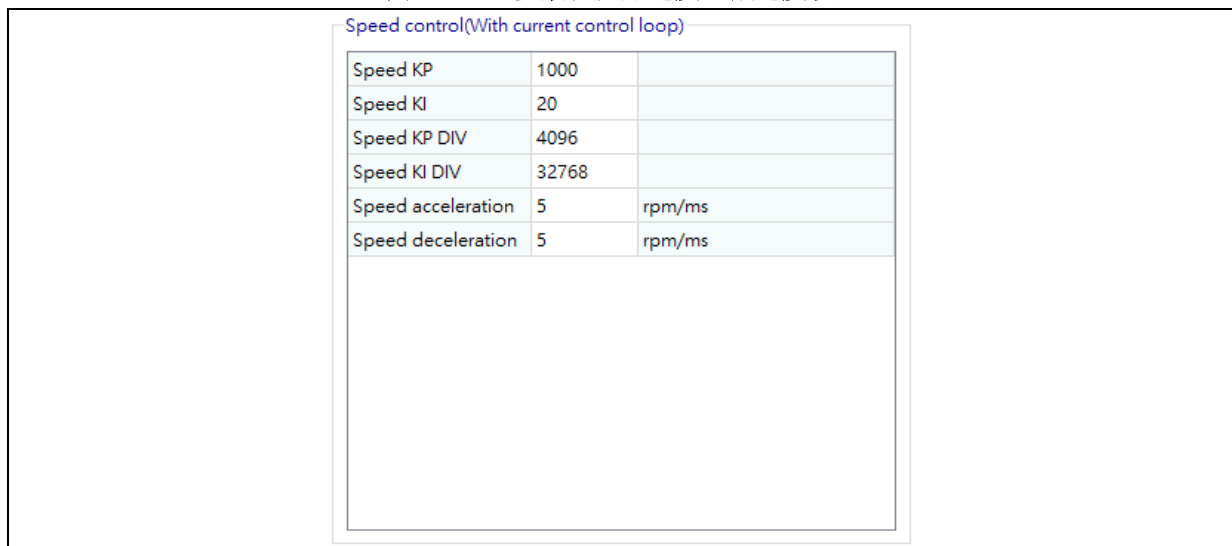
3.8 速度环控制

此模式下可调控的参数有速度环的 PID 参数以及加速度、减速度，调整完成后亦可由波形绘制查看响应，详细操作步骤如下：

STEP-1. 将控制模式下拉菜单选为 Speed Control

STEP-2. 切换到 Tuning Parameters 页面，设置速度 PID 参数以及加速度、减速度

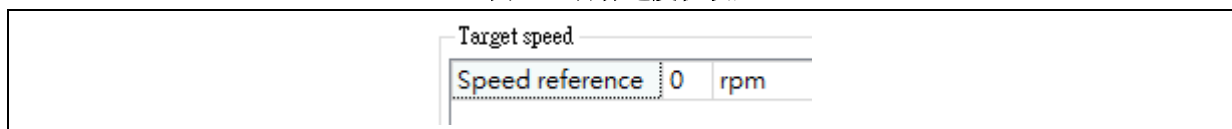
图 27. PID 参数以及加速度、减速度设置



Speed control(With current control loop)		
Speed KP	1000	
Speed KI	20	
Speed KP DIV	4096	
Speed KI DIV	32768	
Speed acceleration	5	rpm/ms
Speed deceleration	5	rpm/ms

STEP-3. 设置 Control source 为 software control 及目标速度参考值(Speed reference)

图 28. 目标速度值设置



Target speed

Speed reference 0 rpm

STEP-4. 按下启动电机(Start Motor)按钮

STEP-5. 调整绘图区的参数监控为 Speed reference 以及 Speed measured

图 29. 调整信道监控参数(速度环调试)

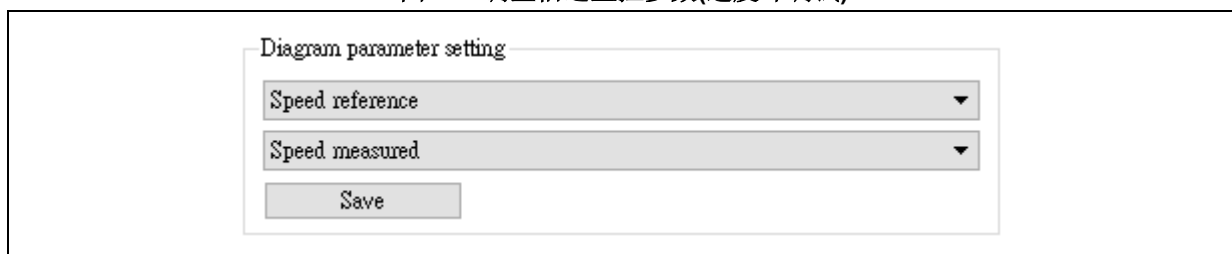


Diagram parameter setting

Speed reference ▼

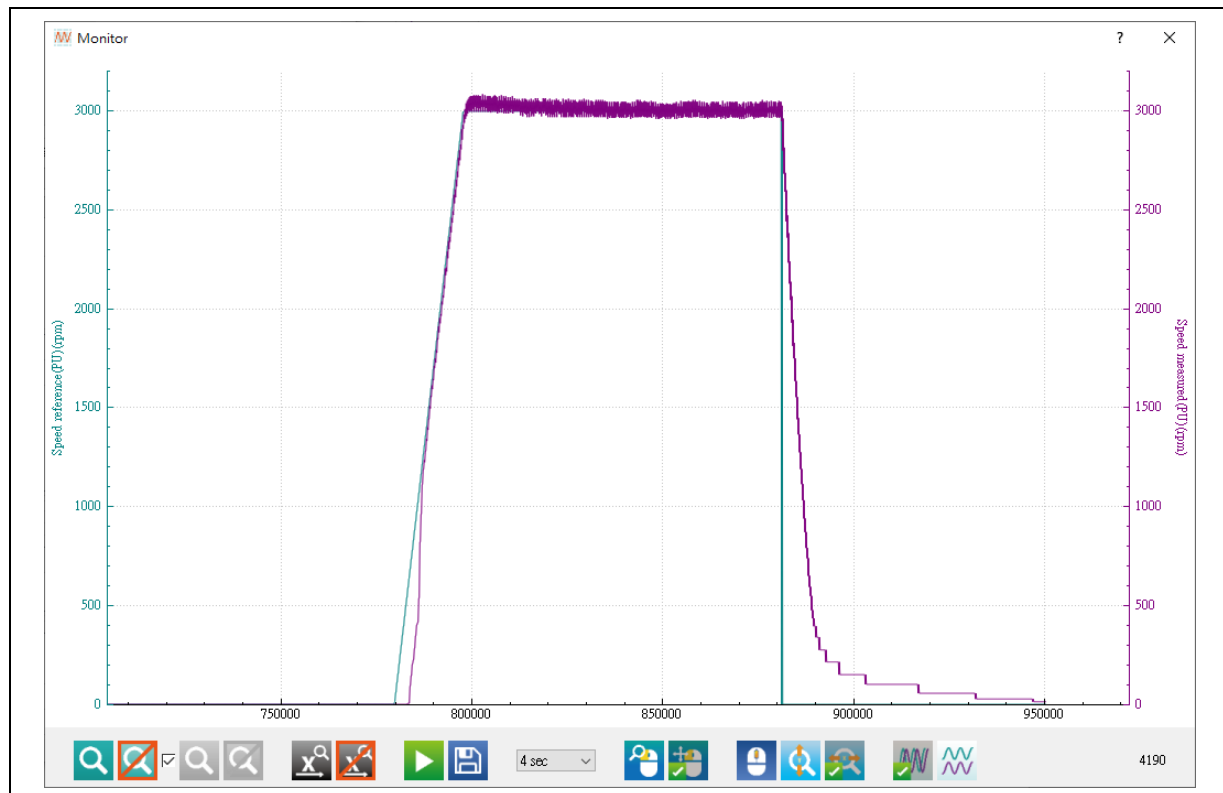
Speed measured ▼

Save

STEP-6. 点选绘图组  即可显示出波形窗口

STEP-7. 查看速度响应是否如预期，如图 30，若不如预期则按下停止电机，并重复 STEP-2~STEP-

图 30. 速度环调试波形



3.9 速度电压环调试

由于低转速时的电流非常小，使用电流控制时较难控制，因此本范例工程提供低速电压的控制方法，在低转速时使用电压控制，直到电流较大时切换回电流控制闭回路控制。

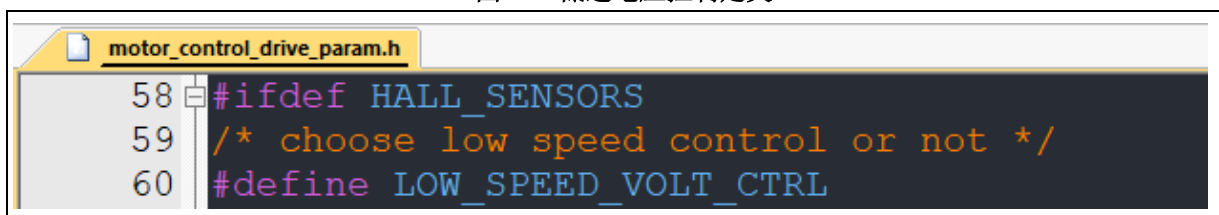
速度电压环为根据速度误差值对电压输出进行控制，因此若开启纯电压控制的定义

WITHOUT_CURRENT_CTRL，或是开启低速电压控制的定义 LOW_SPEED_VOLT_CTRL 的低速区间时，会使用到速度电压环。

此模式下可调的参数有速度电压环的 PID 参数，调整完成后亦可由波形绘制查看响应，详细操作步骤如下：

STEP-1. 定义 motor_control_drive_param.h 文件的宏 LOW_SPEED_VOLT_CTRL，如图 31。

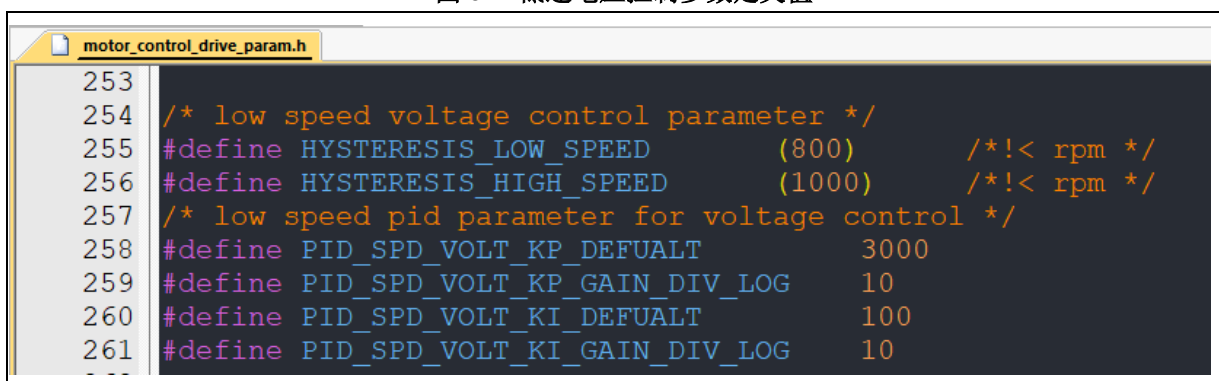
图 31. 低速电压控制定义



```
58 #ifdef HALL_SENSORS
59 /* choose low speed control or not */
60 #define LOW_SPEED_VOLT_CTRL
```

STEP-2. 设置电流控制切换到电压控制的最小速度值(HYSTERESIS_LOW_SPEED)以及电压控制切换到电流控制的最大速度值(HYSTERESIS_HIGH_SPEED)，如图 32。

图 32. 低速电压控制参数定义值

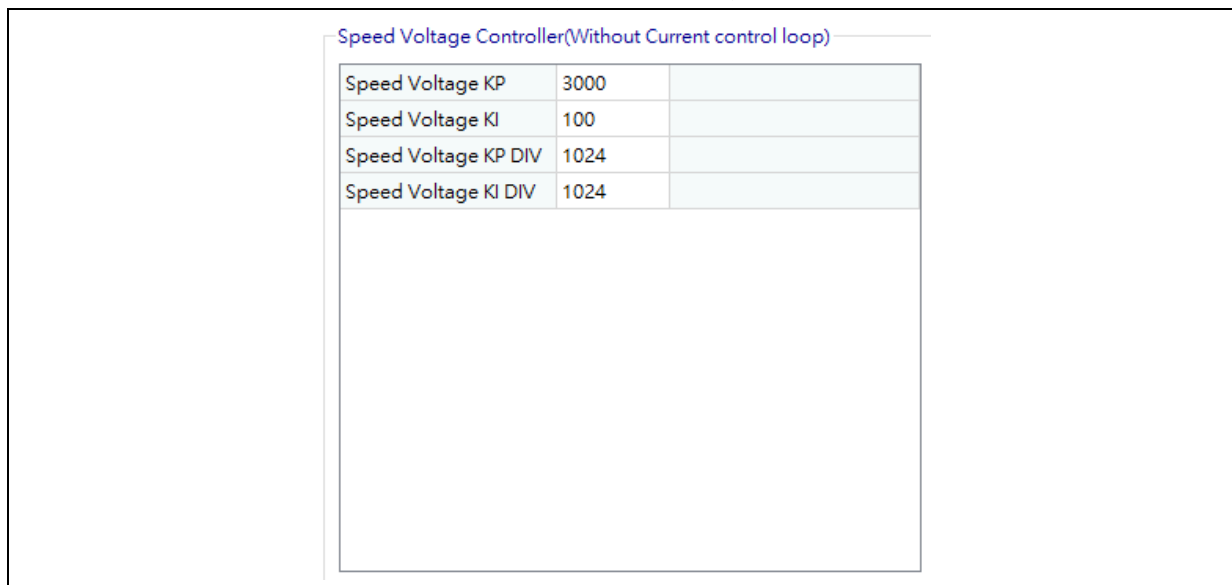


```
253
254 /* low speed voltage control parameter */
255 #define HYSTERESIS_LOW_SPEED      (800)      /*!< rpm */
256 #define HYSTERESIS_HIGH_SPEED     (1000)     /*!< rpm */
257 /* low speed pid parameter for voltage control */
258 #define PID_SPD_VOLT_KP_DEFAULT   3000
259 #define PID_SPD_VOLT_KP_GAIN_DIV_LOG 10
260 #define PID_SPD_VOLT_KI_DEFAULT   100
261 #define PID_SPD_VOLT_KI_GAIN_DIV_LOG 10
```

STEP-3. 将控制模式下拉菜单选为 Speed Control

STEP-4. 切换到 Tuning Parameters 页面，设置速度电压 PID 参数

图 33. 速度电压 PID 参数设置

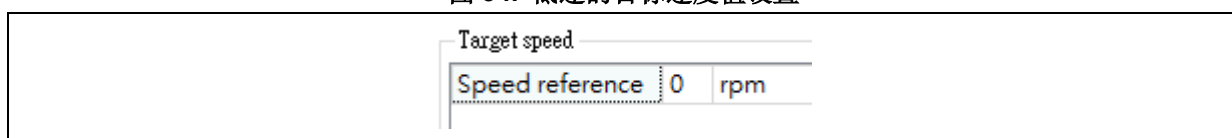


Speed Voltage Controller(Without Current control loop)

Speed Voltage KP	3000	
Speed Voltage KI	100	
Speed Voltage KP DIV	1024	
Speed Voltage KI DIV	1024	

STEP-5. 设置 Control source 为 software control 及低速的目标速度参考值(Speed reference), 如 300 rpm

图 34. 低速的目标速度值设置



Target speed

Speed reference 0 rpm

STEP-6. 按下启动电机(Start Motor)按钮

STEP-7. 调整绘图区的参数监控为 Speed reference(PU)以及 Speed measured(PU) 后按下 Save 按钮。

图 35. 调整通道监控参数(速度环调试)

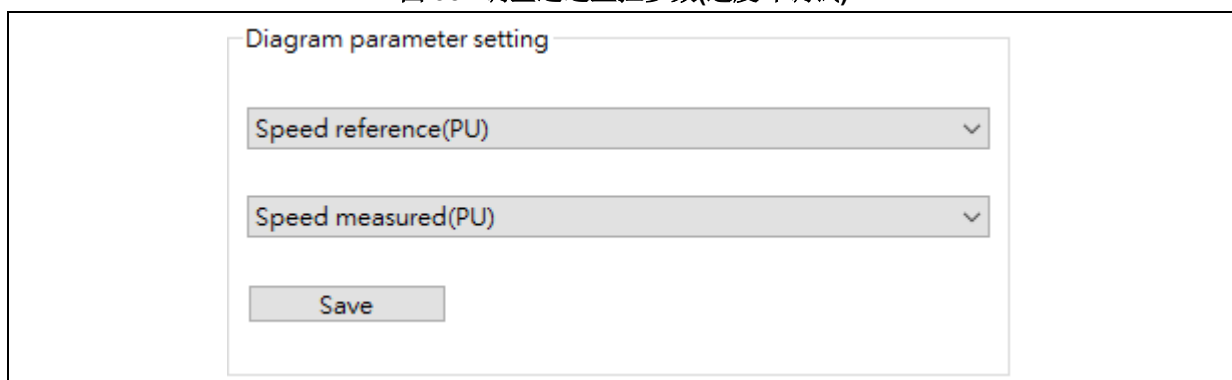


Diagram parameter setting

Speed reference(PU) ▼

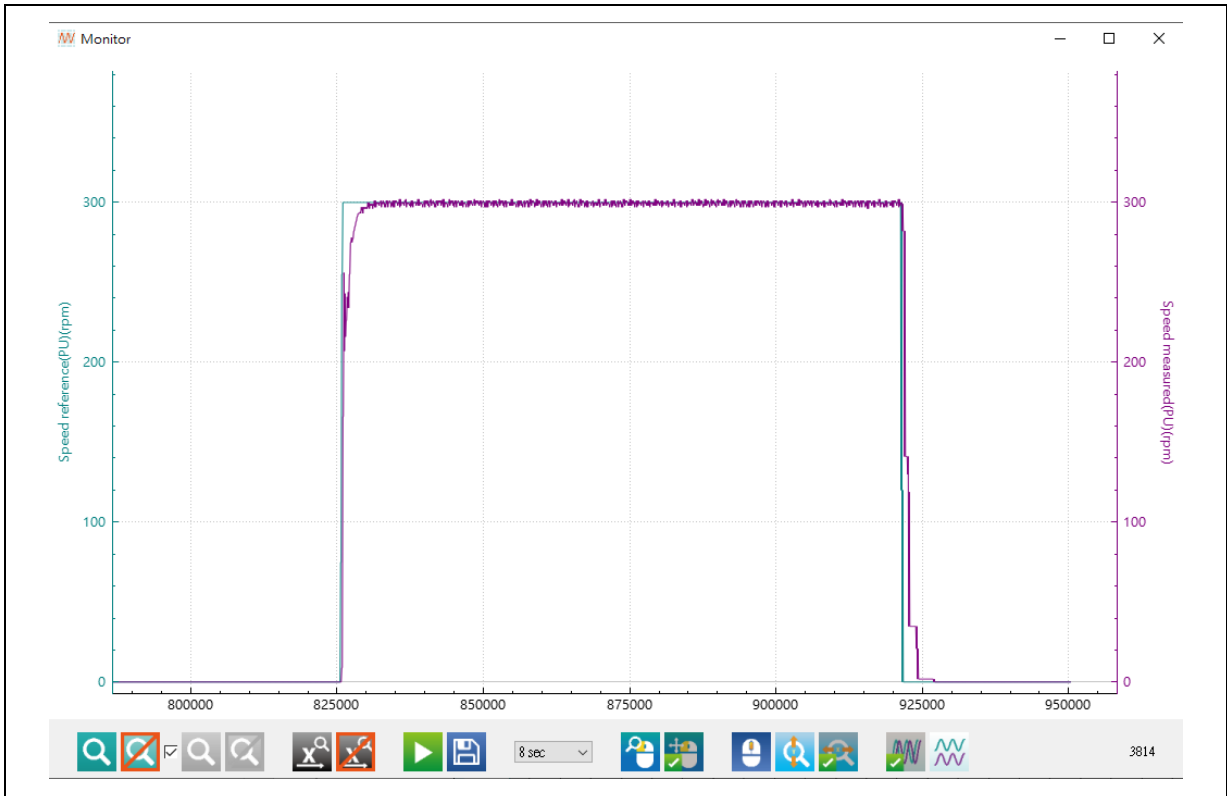
Speed measured(PU) ▼

Save

STEP-8. 点选绘图按钮即可呼叫出波形窗口

STEP-9. 查看速度响应是否如预期, 如图 36, 若不如预期则按下停止电机, 并重复 STEP-4~STEP-9

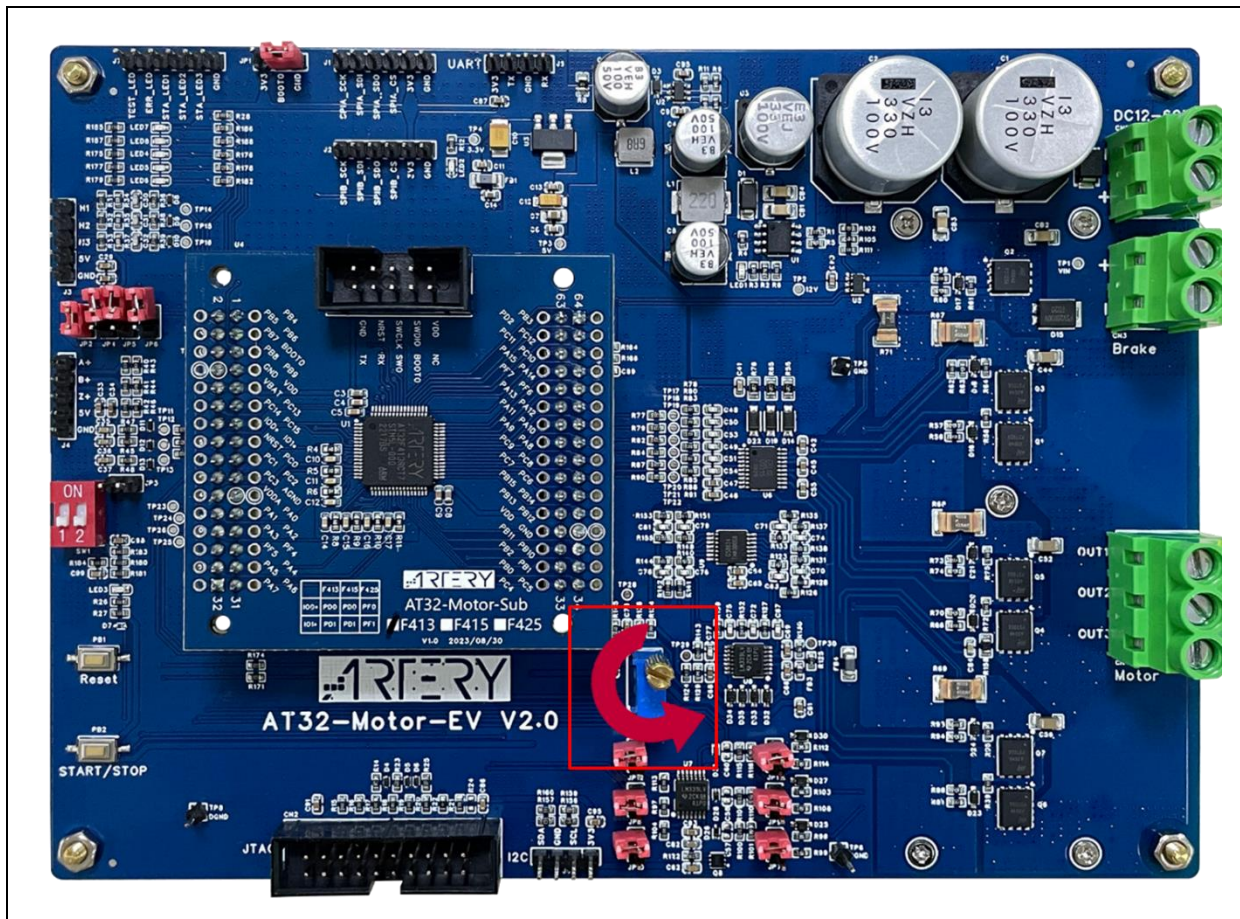
图 36. 速度电压环调试波形



3.10 外部/软件命令控制

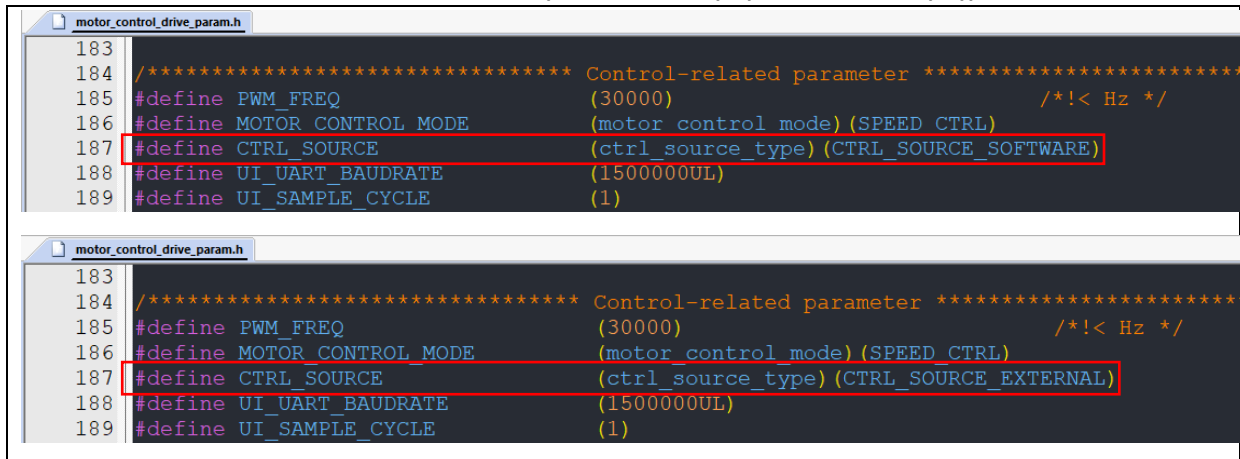
本范例工程支持两种控制来源，包含外部电压控制以及软件控制，外部电压控制为通过外部的电压调整速度或转矩的大小，在本低压电机控制开发板为调整红框处的电位器来控制命令值的大小，电位器为逆时针转动电压会递增，软件控制则为直接输入速度或转矩命令值来控制，本范例工程初始设定默认为软件控制模式。

图 37. 电位器位置图（外部电压控制来源）



控制来源可以直接修改程序定义或是通过电机应用 PC 软件来做修改，程序定义值于 motor_control_drive_param.h 文档内的 CTRL_SOURCE，如图 38。通过 PC 软件修改的修改方式详见图 39，且根据用户选择不同的控制模式会对应不同的控制参数，若为速度控制模式时则会显示目标速度的控制栏位，如图 39，若选择转矩控制则会有目标转矩电流的控制栏位，如图 40。

图 38. 控制来源定义值(软件控制来源(上)/外部电压控制(下))



1) 外部电压控制来源

在来源选择的下拉菜单中若选择 **external voltage control** 则可以切换到外部来源控制，可通过外部的电压调整速度或转矩的大小，目标速度或目标转矩的栏位将会显示目前控制电压下所换算的控制速度或转矩大小。

注意：外部来源控制模式下本栏位不可修改。

图 39. 速度控制模式(外部电压控制来源)

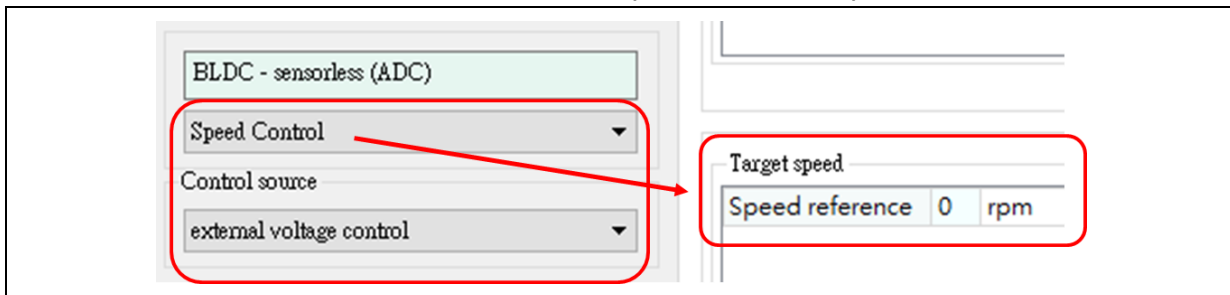
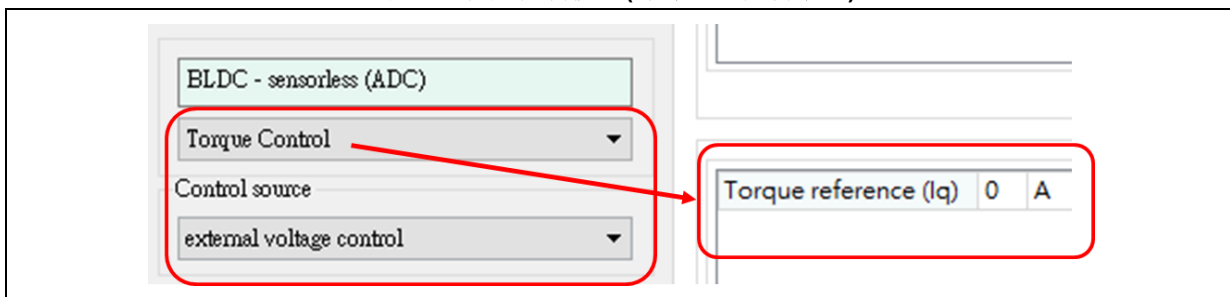


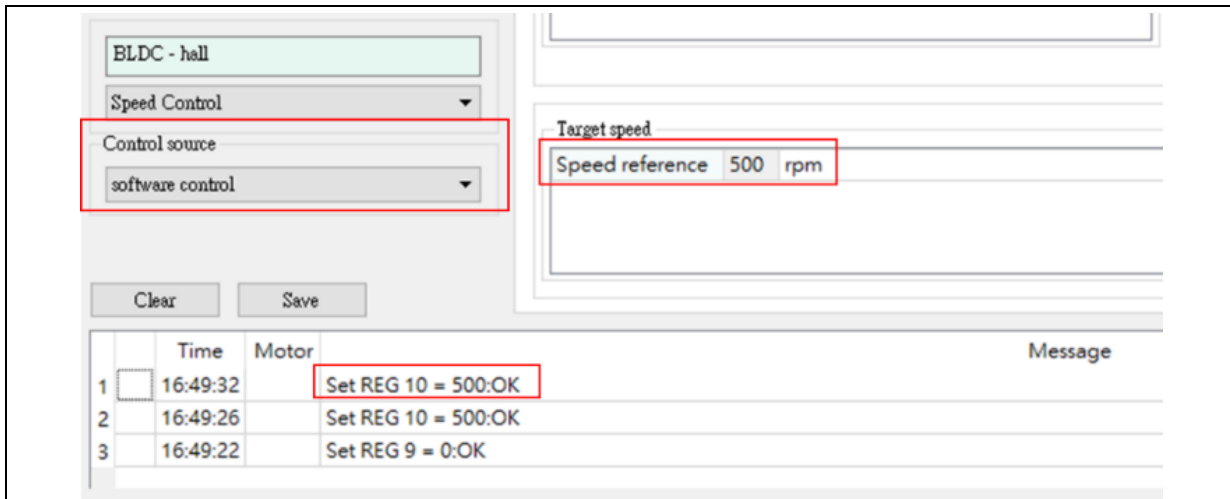
图 40 转矩控制模式(外部电压控制来源)



2) 软件控制来源

默认为软件控制模式，来源选择的下拉菜单中选择 **Software control** 也可切换至软件控制模式，如图 41，在此模式下通过修改 UI 接口的目标速度/转矩来调整电机控制的速度/转矩大小，双击此栏位即可修改数值，设定完成可看到下面栏位显示设置成功的信息。

图 41 软件控制来源设置目标速度



4 文档版本历史

表 14. 文档版本历史

日期	版本	变更
2024.05.17	2.1.0	最初版本
2024.12.27	2.1.1	闪存配置更新、更新速度电压环控制、更新定义
2025.10.09	2.1.2	闪存配置修正

重要通知 - 请仔细阅读

买方自行负责对本文所述雅特力产品和服务的选择和使用，雅特力概不承担与选择或使用本文所述雅特力产品和服务相关的任何责任。

无论之前是否有过任何形式的表示，本文档不以任何方式对任何知识产权进行任何明示或默示的授权或许可。如果本文档任何部分涉及任何第三方产品或服务，不应被视为雅特力授权使用此类第三方产品或服务，或许可其中的任何知识产权，或者被视为涉及以任何方式使用任何此类第三方产品或服务或其中任何知识产权的保证。

除非在雅特力的销售条款中另有说明，否则，雅特力对雅特力产品的使用和 / 或销售不做任何明示或默示的保证，包括但不限于有关适用性、适合特定用途（及其依据任何司法管辖区的法律的对应情况），或侵犯任何专利、版权或其他知识产权的默示保证。

雅特力产品并非设计或专门用于下列用途的产品：（A）对安全性有特别要求的应用，例如：生命支持、主动植入设备或对产品功能安全有要求的系统；（B）航空应用；（C）航天应用或航天环境；（D）武器，且/或（E）其他可能导致人身伤害、死亡及财产损害的应用。如果采购商擅自将其用于前述应用，即使采购商向雅特力发出了书面通知，风险及法律责任仍将由采购商单独承担，且采购商应独力负责在前述应用中满足所有法律和法规要求。

经销的雅特力产品如有不同于本文档中提出的声明和 / 或技术特点的规定，将立即导致雅特力针对本文所述雅特力产品或服务授予的任何保证失效，并且不应以任何形式造成或扩大雅特力的任何责任。

© 2024 雅特力科技 保留所有权利